

Programming Task: Hybrid Quantum-Classical Classification on a Real-World Dataset

How much time to spend: about 3 working days, roughly 24 hours of focused work. We recommend spreading this across 5 to 7 calendar days so you have time to think and design between sessions.

What to hand in: a git repository (GitHub or GitLab, public or private as long as reviewers have access), with a written report committed inside it as Markdown or PDF.

Required framework: [PennyLane](https://pennylane.ai/) (<https://pennylane.ai/>) for the quantum components. You may pair it with any classical ML framework you like (scikit-learn, PyTorch, JAX, etc.); justify your choice briefly.

On using LLMs: free-tier LLMs are fair game for learning quantum concepts and for accelerating boilerplate. If you use them, please include a short paragraph in the report saying how and where. We are interested in the judgement you bring on top of LLM output, not in whether you avoided them.

1. A bit of context

You are interviewing for a role on a team that works across three connected areas:

1. Maintaining a QML library with cleanly abstracted layer interfaces and a fast simulator backend.
2. Checking whether the quantum part of a hybrid model is actually contributing, using bounds, metrics, and architecture search.
3. Looking further ahead towards fault tolerant QML, including T gate cost optimization and discretized quantum circuits.

A short note on where this task sits. Supervised classification is the workhorse setting for most practical QML experiments, and it is where most of the "does quantum help" literature actually lives. The question turns out to be surprisingly subtle. A naive quantum kernel or a poorly chosen feature encoding will often lose to a well-tuned classical baseline on any realistic dataset, and the interesting work sits in the gap between those two failure modes: knowing what encoding to pick for the data you have, and knowing how to measure whether the quantum part is contributing anything beyond what a matched classical model could do on its own.

Your objective: build a small framework for training and evaluating hybrid quantum-classical classifiers, apply it to a published real-world dataset, and figure out with evidence whether the quantum component is doing useful work.

2. The project

2.1 The research question

Take a published real-world classification problem, build a hybrid quantum-classical classifier for it, and determine with evidence whether the quantum

component is contributing anything beyond what a parameter-matched classical model provides.

Everything below is scaffolding around this one question.

2.2 The dataset

Please use the **Breast Cancer Wisconsin (Diagnostic) Dataset** from the UCI Machine Learning Repository, available via `sklearn.datasets.load_breast_cancer()` or directly from UCI.

A few reasons for this choice. It is a real medical dataset (features derived from digitized images of fine-needle aspirates of breast masses), with 569 samples, 30 features, and a binary malignant/benign label. There is a strong published classical baseline (a well-tuned RBF-kernel SVM reaches around 97% test accuracy), so "does the quantum component help" is a genuinely meaningful question rather than a rhetorical one. And there is an active QML benchmarking literature on this specific dataset, so Part B has real prior work to engage with.

One warning worth flagging upfront. This dataset is small, and several published QML papers report suspiciously high test accuracies on it. A big part of what we are evaluating in Part B is whether your evaluation protocol is honest. Proper train/validation/test splits, cross-validation where appropriate, and reporting results across multiple random splits matter more here than squeezing out an extra point of accuracy.

You are welcome to add a second dataset on top if you think it strengthens your conclusions, but Breast Cancer Wisconsin is the required common reference point so that we can compare candidates on equal footing.

2.3 What to build

The project has three parts: a classifier framework (Part A), an empirical study of whether the quantum component helps (Part B), and a fault-tolerance-flavoured probe (Part C).

Everything below is guidance, not a specification. You decide how to split your time across the three parts, and you decide which one to treat as your centre of gravity. We want to see you pick an emphasis deliberately and defend that choice in the report, rather than spreading yourself thin across all three. A submission that goes deep on one part and lighter on the others will do better than one that tries to polish everything to the same level.

The one thing we ask for regardless of your emphasis is basic engineering hygiene. Code we can read and run, tests that mean something, experiments we can reproduce. The depth of the framework, the ambition of the empirical study, and the reach of the fault-tolerance probe are all yours to scale up or down.

Part A: Classifier framework

A small, installable Python package that supports training and evaluating hybrid quantum-classical classifiers on this kind of tabular dataset. It should be designed so that a teammate could plug in a new data encoding, a new ansatz, a new classical pre- or post-processing head, or a new classical baseline without touching the evaluation code.

You can take any of the main hybrid QML approaches for the quantum component (a variational quantum classifier, a quantum kernel method, or something else with justification). The choice is yours and should be defended in the report. Whatever you pick, the framework must support both the quantum model and at least one honest classical baseline under an identical evaluation protocol.

We will read and run this code. Things we pay attention to:

- Module boundaries and class design. Data loading, preprocessing, encoding, the quantum part, the classical head, and evaluation should be separable concerns.
- Type hints, docstrings, error handling.
- Tests that exercise something meaningful, not coverage theatre. A test that checks your quantum kernel matrix is symmetric and positive semidefinite within numerical tolerance is an example of the right flavour.
- Reproducibility from a fresh clone. One documented command should set up the environment, and one documented command per experiment should reproduce the corresponding figure in your report.
- Config-driven experiments. Hyperparameters hard-coded inside scripts are a red flag.
- Git hygiene. Readable commits, no `final_v2_FINAL.ipynb`.

If Part A is your centre of gravity, something worth thinking about: switching between the quantum classifier and the classical baseline should be a configuration change, not a code change. If running the baseline requires a separate script, your abstractions are probably not where they need to be.

Part B: The research question

Apply your framework to the dataset and investigate whether the quantum component contributes anything meaningful.

As a guideline, a complete treatment would include:

1. **Careful data preprocessing.** 30 features will not fit on a small simulator directly. How you reduce or project the features is a design choice that matters a lot and should be defended in the report. Explain what you did, what alternatives you considered, and why.
2. **Thoughtful encoding design.** How you embed the (preprocessed) classical feature vector into a quantum state is arguably the single most consequential choice in the whole pipeline, because the encoding determines what the quantum part can represent at all. Try more than one encoding if you have time, and explain what you learn from the comparison.
3. **At least one parameter-matched classical baseline** trained under the same protocol. The baseline must be tuned (hyperparameter search over regularization, kernel bandwidth, etc.), or the comparison is not honest. A well-tuned RBF-SVM is the obvious choice; you are welcome to add others.
4. **At least two quantitative measures of "usefulness"** beyond raw test accuracy. Accuracy is noisy at this dataset size and can hide a lot. Defining richer metrics is part of the task; defend your choices in the report.
5. **An architecture study** across at least one design axis of the quantum model (encoding family, ansatz depth, qubit count, measurement strategy, whichever you pick). Keep it tractable; a grid of a handful of configurations is fine.
6. **Proper experimental hygiene.** Multiple random train/test splits, stratified cross-validation, and honest error bars. Single-split results are not informative for this part, and this dataset in particular is small enough that single-split results can be actively misleading.
7. **A clear, defensible conclusion.** A well-argued "the quantum component does not help on this dataset under these conditions, and here is my hypothesis for why" is an excellent outcome. Overselling a null result is not, and neither is reporting 100% test accuracy without a hard look at whether that number means what you think it means.

We are not hinting at what encodings, metrics, or architectures to use. Choosing them is the test. Read the literature, form an opinion, commit to it, and defend it.

Part C: Fault-tolerance-flavoured probe

Take the best-performing hybrid configuration from Part B and deliver a short, focused investigation of what it would cost to run this model in a fault-tolerant setting, or of what happens when you constrain it to be more fault-tolerance-friendly.

Pick one of the following and deliver a short, focused investigation:

- **C1: T-gate counting.** Decompose the trained circuit into a Clifford plus T gate set, report the T -count and T -depth, and discuss what this would mean for the cost of running this model on a fault-tolerant machine. *Bonus:* try one simple strategy to reduce the T -count and report how classification performance trades off against T -count.
- **C2: Discrete-parameter QML.** Retrain the best configuration with its variational parameters constrained to a discrete set. Compare its performance to the continuous-parameter version and discuss what this says about fault-tolerant compilation.

Part C does not need to be polished research. We want to see that, given a new QC topic and a few hours, you can read around, prototype something, and form a defensible opinion.

2.4 What we are not asking for

- State-of-the-art accuracy on Breast Cancer Wisconsin. A well-executed study that lands at 95% with honest error bars beats a 100% submission that evaluated on the training set.
- Novel quantum algorithms.
- Large qubit counts. Stay in a simulator-friendly regime; 4 to 8 qubits is appropriate after preprocessing.
- UIs, dashboards, or deployed services.
- More hours than the time budget. If you are at 30 hours and not done, please stop and write up what you have.

3. What to submit

3.1 The repository

Whichever part you emphasized, we are looking for:

- All the code needed to reproduce every figure and every number in your report. The layout is up to you (installable package, flat scripts, notebooks, or some mix) depending on what you chose to build.
- Tests that exercise the non-trivial parts of your code. We will run `pytest` and expect it to pass.
- Experiment configurations separated from code. How you organize them is your call.
- A `README.md` with setup instructions, how to run things, and how to reproduce each figure.
- `report.md` or `report.pdf`, committed to the repo.
- A dependency manifest with versions pinned or upper-bounded.

If you went deep on Part A, we will expect a cleanly structured installable package. If you went deep on Part B or Part C, a lighter code layout is fine; the evidence of your engineering judgement will come from the code we do see.

3.2 The report (roughly 8 to 12 pages including figures)

Organize it however you like, but it should cover:

1. **Framing.** How you read the research question, which part (A, B, or C) you picked as your centre of gravity, how you split your time across the three parts, and why.
2. **Framework design.** Your key abstractions, what you made extensible, what you deliberately did not abstract.
3. **Data handling.** Your preprocessing, feature reduction, and encoding choices, with the alternatives you considered and the reasons behind your final choices.
4. **Experimental setup.** Ansätze or kernel families, classical baselines, hyperparameters, cross-validation protocol, seeds, compute used.
5. **Usefulness metrics.** What you chose, why, what their limitations are, and what else you would want to measure with more time.
6. **Results.** With error bars across splits and a clear, calibrated answer to the research question.
7. **Fault-tolerance probe.** What you did in Part C, what you found, what it implies.
8. **Limitations and next steps.** What would change your conclusion? What would you build next?
9. **LLM usage disclosure.**

A good report is self-contained, honest about limitations, and curated. It should not read like a chronological lab notebook.

4. What we are looking for

Area	What good looks like
Software engineering	Clean OOP where applicable, sensible abstractions, meaningful tests, reproducibility, git hygiene. We judge this against whatever code you actually wrote.
Picking up a new topic	How quickly and how correctly you come up to speed on PennyLane and quantum concepts from a cold start.
Problem scoping	How you cut the work to fit the budget, what you chose to test rigorously versus skim, and how you defended those cuts.
Problem solving and critical thinking	The quality of your encoding and preprocessing choices, your metric design beyond raw accuracy, the rigour of your evaluation protocol, and the calibration of your conclusions.

Area	What good looks like
Reporting	Clarity, structure, and how well your claims match your evidence.
Quantum computing	Correct use of PennyLane, reasonable encoding and ansatz design, meaningful engagement with Part C.
Classical ML	A working classification pipeline, sensible baseline tuning, proper cross-validation, awareness of the statistical issues that come with small datasets.

There are no fixed weights. Every area above has at least one clear place in the task where you can show what you can do. You are not expected to be equally strong on all of them today; we are particularly interested in how you handle the areas that are furthest from your background.

5. Rules of the road

Time. Around 24 hours of focused work, across 3 working days. Spreading it over a calendar week is fine and encouraged. If you are past 30 hours, please stop and write up. Knowing when to stop is part of the skill.

Questions. If something is genuinely blocking you, email us. It is also fine to write "I assumed X because the spec was ambiguous, and here is why" in the report and carry on.

Scope cuts. Cuts are expected and will not be held against you as long as you document them. If you cut something, say so in the report and say what you would have done.

Attribution. Please cite any paper, blog post, tutorial, or piece of code you adapted. Adapting is fine; uncredited copying is not. There is existing published work on QML classifiers for Breast Cancer Wisconsin, and you are expected to be aware of it, cite what you use, and be clear about what is your own contribution versus what you reproduced or adapted.

Submission. Send us the repo link (invite `<reviewer-handles>` if it is private) and tell us which commit SHA you would like us to review.

Looking forward to reading your work. Good luck.